data transformation with R and dplyr

hands-on data science · prof. dr. nicolas meseth

1 why transform
2 the verbs
3 the pipe

no cook serves the groceries the way they come from the market, and every recipe is a handful of techniques in a certain order.

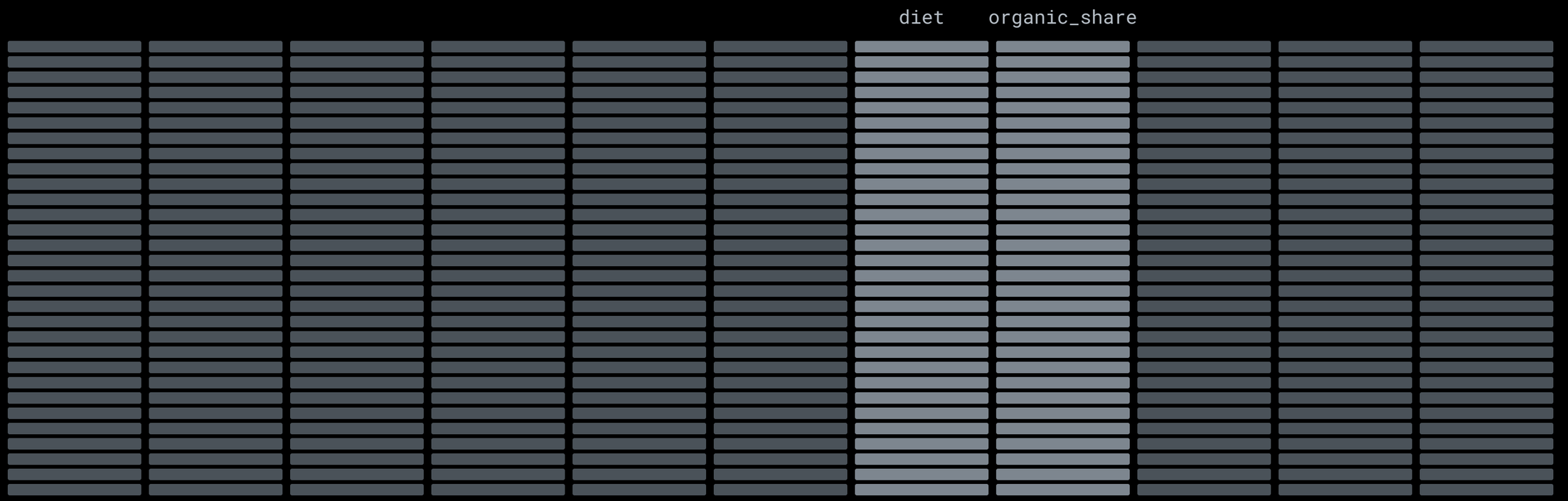


image: ai-generated (gpt-image-2.5-flare)

part 1

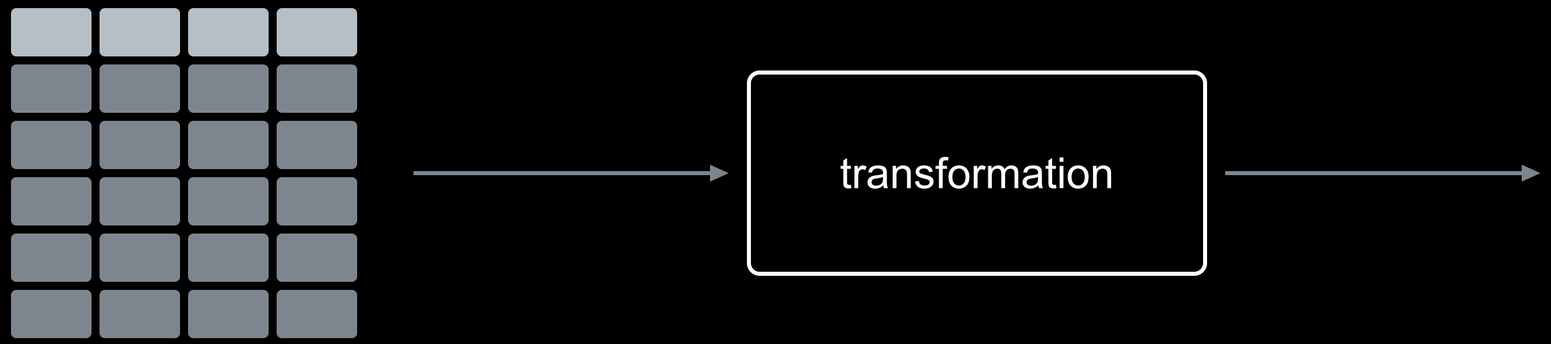
why transform

do vegetarians buy more organic food than others?



the answer is in the table, but you cannot read it off.

a table in, a table out



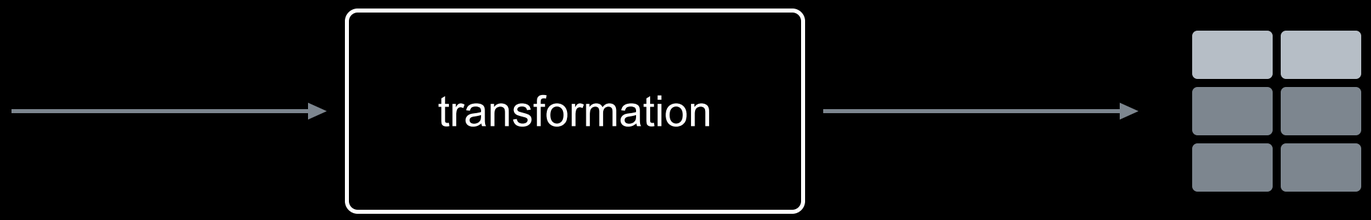
every transformation takes a table and returns a table, which is why they can be chained.

a table in, a table out



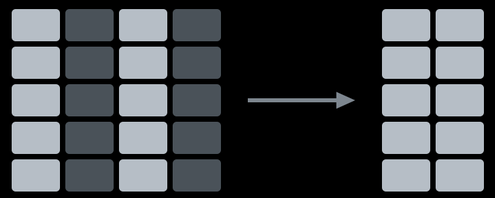
every transformation takes a table and returns a table, which is why they can be chained.

a table in, a table out



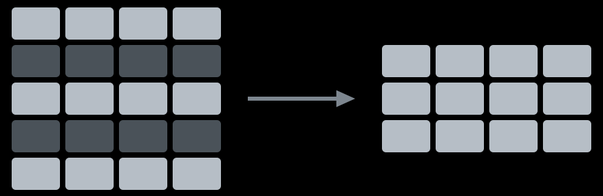
every transformation takes a table and returns a table, which is why they can be chained.

six basic operations



select columns

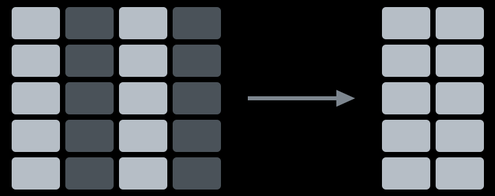
which columns do i need?



filter rows

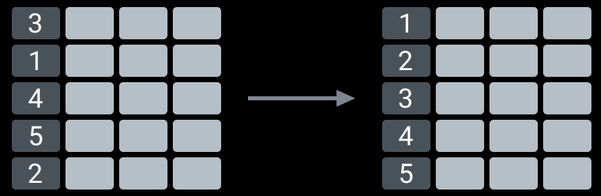
who is this about?

six basic operations



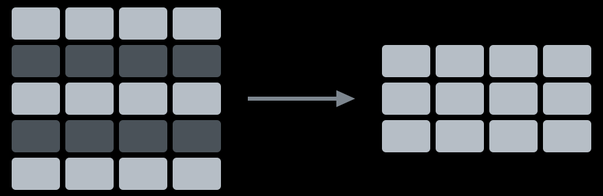
select columns

which columns do i need?



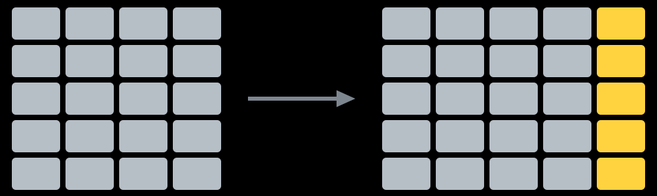
sort

what comes first?



filter rows

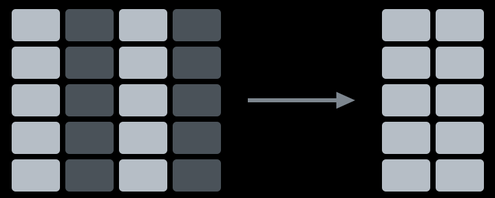
who is this about?



compute new columns

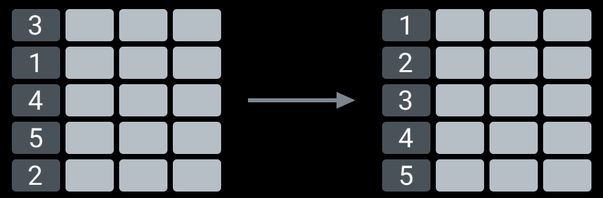
what can be derived?

six basic operations



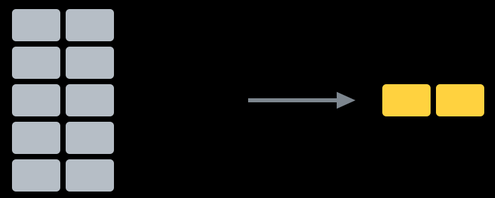
select columns

which columns do i need?



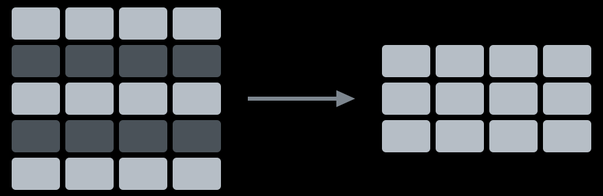
sort

what comes first?



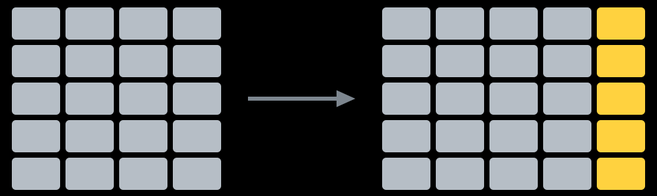
summarize

what is typical, how many?



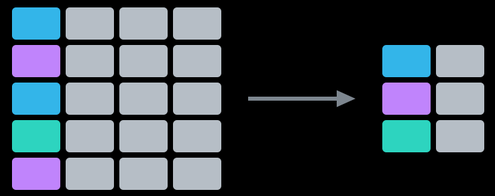
filter rows

who is this about?



compute new columns

what can be derived?



group

how do subgroups differ?

the same ideas everywhere

operation	dplyr	SQL	pandas	excel
select columns	select()	SELECT	df[columns]	hide columns
filter rows	filter()	WHERE	df.query()	filter
sort	arrange()	ORDER BY	sort_values()	sort
new columns	mutate()	SELECT … AS	df.assign()	formula column
summarize	summarize()	COUNT, AVG	df.agg()	pivot table
group	group_by()	GROUP BY	df.groupby()	pivot table

part 2

the verbs

dplyr names its functions after what they do

```
library(dplyr)
```

```
library(dplyr)
```

the first argument is always a table

```
library(dplyr)
```

the first argument is always a table
columns go by their name, without quotes and without $

```
library(dplyr)
```

the first argument is always a table
columns go by their name, without quotes and without $
the result is a new table, the original stays as it was

six rows of the survey

id	age	diet	organic_share
1	56	omnivore	2
2	55	omnivore	2
3	47	omnivore	1
4	32	omnivore	1
5	74	flexitarian	2
6	48	vegetarian	2

organic_share: share of organic food when shopping, from 1 (hardly any) to 5 (almost all)

```
select(survey, id, diet)
```

id	age	diet	organic_share
1	56	omnivore	2
2	55	omnivore	2
3	47	omnivore	1
4	32	omnivore	1
5	74	flexitarian	2
6	48	vegetarian	2

```
select(survey, id, diet)
```

id	age	diet	organic_share
1	56	omnivore	2
2	55	omnivore	2
3	47	omnivore	1
4	32	omnivore	1
5	74	flexitarian	2
6	48	vegetarian	2

```
select(survey, id, diet)
```

id	diet
1	omnivore
2	omnivore
3	omnivore
4	omnivore
5	flexitarian
6	vegetarian

three ways to say which columns

```
select(survey, id, diet)
```

by name

```
select(survey, -organic_share)
```

all but this one

```
select(survey, id, starts_with("organic"))
```

by a pattern in the name

```
filter(survey, age >= 50)
```

id	age	diet	organic_share
1	56	omnivore	2
2	55	omnivore	2
3	47	omnivore	1
4	32	omnivore	1
5	74	flexitarian	2
6	48	vegetarian	2

```
filter(survey, age >= 50)
```

id	age	diet	organic_share	
1	56	omnivore	2	
2	55	omnivore	2	
3	47	omnivore	1	
4	32	omnivore	1	
5	74	flexitarian	2	
6	48	vegetarian	2	

```
filter(survey, age >= 50)
```

id	age	diet	organic_share	
1	56	omnivore	2	
2	55	omnivore	2	
5	74	flexitarian	2	

```
filter(survey, diet == "vegan")
filter(survey, age >= 50 & diet != "omnivore")
filter(survey, diet %in% c("vegan", "vegetarian"))
filter(survey, !is.na(organic_share))
```

`==`	equal
`!=`	not equal
`&`	and
`|`	or
`%in%`	one of
`is.na()`	missing

= assigns, == compares

```
arrange(survey, desc(age))
```

id	age	diet	organic_share
1	56	omnivore	2
2	55	omnivore	2
3	47	omnivore	1
4	32	omnivore	1
5	74	flexitarian	2
6	48	vegetarian	2

```
arrange(survey, desc(age))
```

id	age	diet	organic_share
5	74	flexitarian	2
1	56	omnivore	2
2	55	omnivore	2
6	48	vegetarian	2
3	47	omnivore	1
4	32	omnivore	1

```
arrange(survey, age)
arrange(survey, desc(age))
arrange(survey, diet, desc(age))
```

```
mutate(survey, decade = age %/% 10 * 10)
```

id	age	diet	organic_share
1	56	omnivore	2
2	55	omnivore	2
3	47	omnivore	1
4	32	omnivore	1
5	74	flexitarian	2
6	48	vegetarian	2

```
mutate(survey, decade = age %/% 10 * 10)
```

id	age	diet	organic_share	decade
1	56	omnivore	2	
2	55	omnivore	2	
3	47	omnivore	1	
4	32	omnivore	1	
5	74	flexitarian	2	
6	48	vegetarian	2	

```
mutate(survey, decade = age %/% 10 * 10)
```

id	age	diet	organic_share	decade
1	56	omnivore	2	50
2	55	omnivore	2	50
3	47	omnivore	1	40
4	32	omnivore	1	30
5	74	flexitarian	2	70
6	48	vegetarian	2	40

```
mutate(
  survey,
  decade = age %/% 10 * 10,
  organic_fan = organic_share >= 4
)
```

```
summarize(survey, n = n(), mean_age = mean(age))
```

id	age	diet	organic_share
1	56	omnivore	2
2	55	omnivore	2
3	47	omnivore	1
4	32	omnivore	1
5	74	flexitarian	2
6	48	vegetarian	2

```
summarize(survey, n = n(), mean_age = mean(age))
```

id	age	diet	organic_share
1	56	omnivore	2
2	55	omnivore	2
3	47	omnivore	1
4	32	omnivore	1
5	74	flexitarian	2
6	48	vegetarian	2

n
6

```
summarize(survey, n = n(), mean_age = mean(age))
```

n	mean_age
6	52

```
summarize(
  survey,
  n = n(),
  mean_organic = mean(organic_share, na.rm = TRUE),
  oldest = max(age)
)
```

`n()`	number of rows
`mean()`	average
`median()`	middle value
`min()`	smallest
`max()`	largest
`sd()`	spread

one missing value makes the mean NA, unless `na.rm = TRUE`

```
group_by(survey, diet)
```

id	age	diet	organic_share
1	56	omnivore	2
2	55	omnivore	2
3	47	omnivore	1
4	32	omnivore	1
5	74	flexitarian	2
6	48	vegetarian	2

```
group_by(survey, diet)
```

```
# Groups: diet [3]
```

id	age	diet	organic_share
1	56	omnivore	2
2	55	omnivore	2
3	47	omnivore	1
4	32	omnivore	1
5	74	flexitarian	2
6	48	vegetarian	2

```
group_by(survey, diet)
```

```
# Groups: diet [3]
```

id	age	diet	organic_share
1	56	omnivore	2
2	55	omnivore	2
3	47	omnivore	1
4	32	omnivore	1
5	74	flexitarian	2
6	48	vegetarian	2

group_by() and summarize(): one row per group

```
# Groups: diet [3]
```

id	age	diet	organic_share
1	56	omnivore	2
2	55	omnivore	2
3	47	omnivore	1
4	32	omnivore	1
5	74	flexitarian	2
6	48	vegetarian	2

group_by() and summarize(): one row per group

diet	n	mean_age
flexitarian	1	74
omnivore	4	47.5
vegetarian	1	48

```
summarize(
  group_by(survey, diet),
  n = n(),
  mean_age = mean(age)
)
```

group_by() alone changes nothing you can see; it tells the next verb to work per group.

you only want the respondents from Bavaria. which verb?

select()

filter()

arrange()

mutate()



live.hsos.uber.space/q

you only want the respondents from Bavaria. which verb?

select()

filter()

arrange()

mutate()

live.hsos.uber.space/q

you only want the respondents from Bavaria. which verb?

select()

filter()

arrange()

mutate()



live.hsos.uber.space/q

you need the average age per diet. what do you use?

summarize()

group_by()

group_by() + summarize()

mutate()

live.hsos.uber.space/q

you need the average age per diet. what do you use?

summarize()

group_by()

group_by() + summarize()

mutate()

live.hsos.uber.space/q

you need the average age per diet. what do you use?

summarize()

group_by()

group_by() + summarize()

mutate()

live.hsos.uber.space/q

part 3

the pipe

nested calls are read from the inside out

```
arrange(
  summarize(
    group_by(
      filter(survey, age >= 40),
      diet
    ),
    n = n()
  ),
  desc(n)
)
```

nested calls are read from the inside out

```
arrange(
  summarize(
    group_by(
      filter(survey, age >= 40),
      diet
    ),
    n = n()
  ),
  desc(n)
)
```

nested calls are read from the inside out

```
   arrange(
     summarize(
2      group_by(
1        filter(survey, age >= 40),
         diet
       ),
       n = n()
     ),
     desc(n)
   )
```

nested calls are read from the inside out

```
   arrange(
3    summarize(
2      group_by(
1        filter(survey, age >= 40),
         diet
       ),
       n = n()
     ),
     desc(n)
   )
```

nested calls are read from the inside out

```
4  arrange(
3    summarize(
2      group_by(
1        filter(survey, age >= 40),
         diet
       ),
       n = n()
     ),
     desc(n)
   )
```

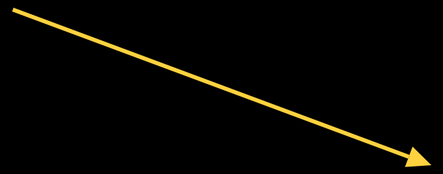
```
older <- filter(survey, age >= 40)
grouped <- group_by(older, diet)
counts <- summarize(grouped, n = n())
result <- arrange(counts, desc(n))
```

readable, but full of names nobody needs again.

the pipe passes the table on

you write

```
x |> f(y)
```



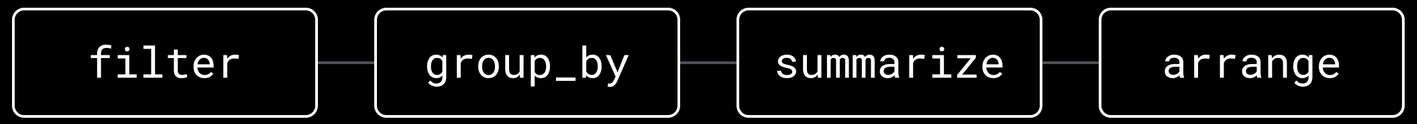
R runs

```
f(x, y)
```

read |> as "and then"

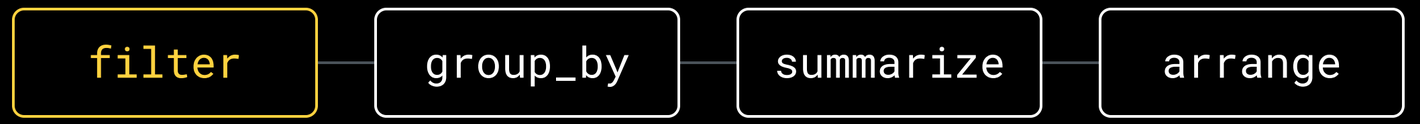
%>% from older code means the same; ctrl+shift+m types the pipe

```
survey |>
  filter(age >= 40) |>
  group_by(diet) |>
  summarize(n = n()) |>
  arrange(desc(n))
```



age	diet
56	omnivore
55	omnivore
47	omnivore
32	omnivore
74	flexitarian
48	vegetarian

```
survey |>
  filter(age >= 40) |>
  group_by(diet) |>
  summarize(n = n()) |>
  arrange(desc(n))
```



age	diet
56	omnivore
55	omnivore
47	omnivore
74	flexitarian
48	vegetarian

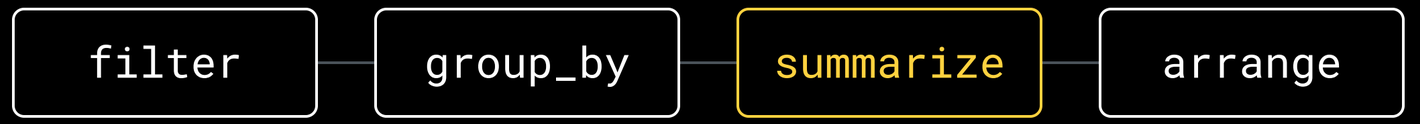
```
survey |>
  filter(age >= 40) |>
  group_by(diet) |>
  summarize(n = n()) |>
  arrange(desc(n))
```




```
# Groups: diet [3]
```

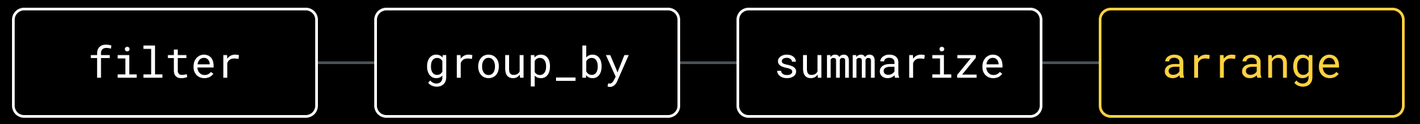
age	diet
56	omnivore
55	omnivore
47	omnivore
74	flexitarian
48	vegetarian

```
survey |>
  filter(age >= 40) |>
  group_by(diet) |>
  summarize(n = n()) |>
  arrange(desc(n))
```



diet	n
flexitarian	1
omnivore	3
vegetarian	1

```
survey |>
  filter(age >= 40) |>
  group_by(diet) |>
  summarize(n = n()) |>
  arrange(desc(n))
```



diet	n
omnivore	3
flexitarian	1
vegetarian	1

```
survey |>
  filter(age >= 40) |>
  group_by(diet) |>
  summarize(n = n()) |>
  arrange(desc(n))
```

the same, as you would write it

```
survey |>
  filter(age >= 40) |>
  group_by(diet) |>
  summarize(n = n()) |>
  arrange(desc(n))
```

take the survey,

```
survey |>
  filter(age >= 40) |>
  group_by(diet) |>
  summarize(n = n()) |>
  arrange(desc(n))
```

take the survey,

and then keep everyone aged 40 or older,

```
survey |>
  filter(age >= 40) |>
  group_by(diet) |>
  summarize(n = n()) |>
  arrange(desc(n))
```

take the survey,

and then keep everyone aged 40 or older,

and then group by diet,

```
survey |>
  filter(age >= 40) |>
  group_by(diet) |>
  summarize(n = n()) |>
  arrange(desc(n))
```

take the survey,

and then keep everyone aged 40 or older,

and then group by diet,

and then count,

```
survey |>
  filter(age >= 40) |>
  group_by(diet) |>
  summarize(n = n()) |>
  arrange(desc(n))
```

take the survey,

and then keep everyone aged 40 or older,

and then group by diet,

and then count,

and then sort by the count.

take the survey
keep the rows with an organic share
split by diet
average and count per group
sort by the average

take the survey
keep the rows with an organic share
split by diet
average and count per group
sort by the average

```
survey
```

take the survey
keep the rows with an organic share
split by diet
average and count per group
sort by the average

```
survey
filter()
```

take the survey
keep the rows with an organic share
split by diet
average and count per group
sort by the average

```
survey
filter()
group_by()
```

take the survey	`survey`
keep the rows with an organic share	`filter()`
split by diet	`group_by()`
average and count per group	`summarize()`
sort by the average	

take the survey	`survey`
keep the rows with an organic share	`filter()`
split by diet	`group_by()`
average and count per group	`summarize()`
sort by the average	`arrange()`

```
survey |>
  filter(!is.na(organic_share)) |>
  group_by(diet) |>
  summarize(
    n = n(),
    mean_organic = mean(organic_share)
  ) |>
  arrange(desc(mean_organic))
```

```
survey |>
  filter(!is.na(organic_share)) |>
  group_by(diet) |>
  summarize(
    n = n(),
    mean_organic = mean(organic_share)
  ) |>
  arrange(desc(mean_organic))
```

diet	n	mean_organic
vegan	1	5
flexitarian	9	2.89
omnivore	17	2.12
vegetarian	2	1.5
pescetarian	1	1

`select()`	columns
`filter()`	rows
`arrange()`	order
`mutate()`	new columns
`summarize()`	one row
`group_by()`	per group
`|>`	and then

to practise: the learning path "data transformation with R and dplyr" on hands-on data science.



image: ai-generated (gpt-image-2.5-flare)